IPD440 - APRENDIZAJE DE MAQUINAS

Clase 5: Kernel Machines

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

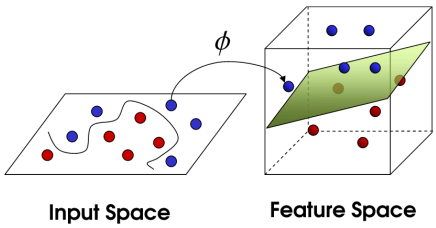
Reformular modelos lineales

En un modelo lineal

$$y(w,x) = w_0 + w_1x_1 + \ldots + w_px_p = w_0 + \mathbf{w}\cdot\mathbf{x}$$

Idea: Extender este modelo lineal a uno no lineal, transformando el espacio de entrada a través de una función $\phi(\cdot)$:

$$w_0 + \mathbf{w}\cdot\phi(\mathbf{x})$$



Kernel

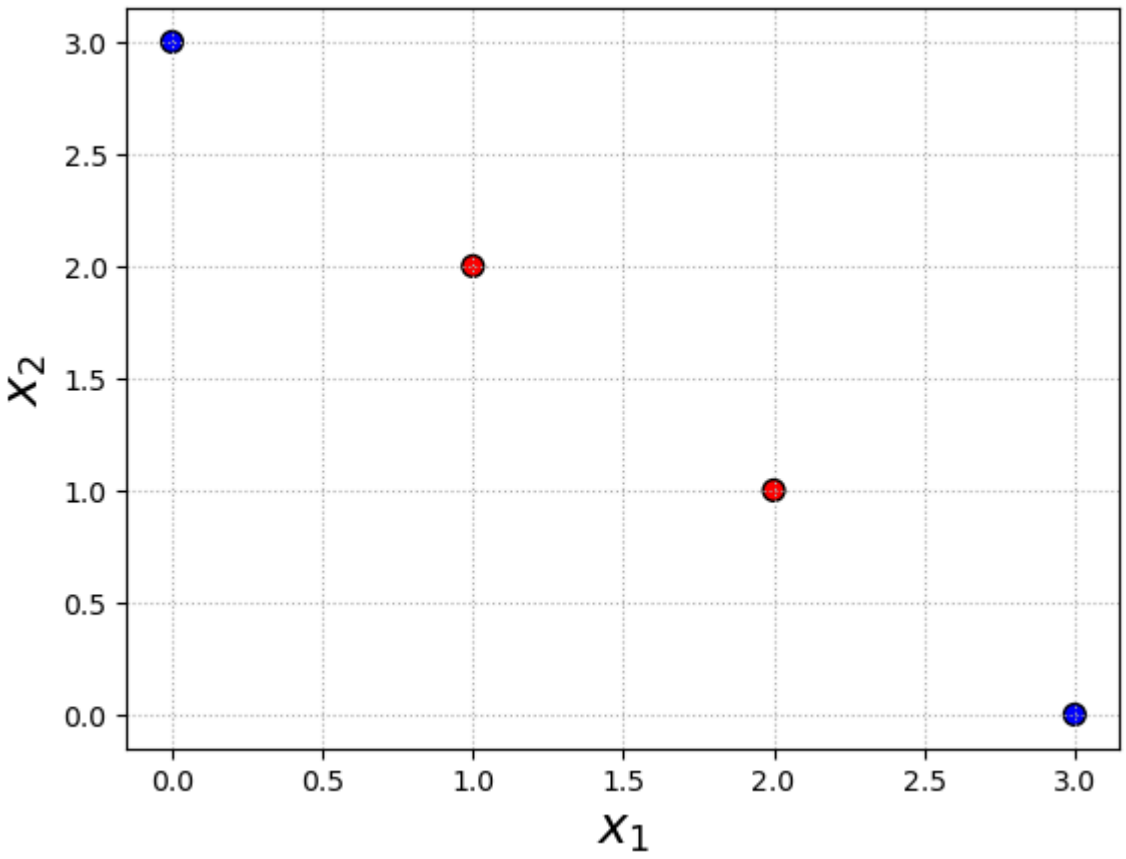
Kernel, informalmente hablando, es un producto interno generalizado entre instancias en el espacio de entrada. Al igual que el producto interno, una función kernel K: $\mathcal{X} \times \mathcal{X} \to \mathbb{R}$ caracteriza propiedades geométricas como la norma, la distancia y el ángulo del espacio de entrada.

Una función $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ es un kernel si existe un perfil $k : [0, \infty[\to \mathbb{R}$, tal que $K(x) = k(\|x\|^2)$.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

In [2]:
```
x_1 = np.array([0,1,2,3])
x_2 = np.array([3,2,1,0])
y = np.array([0,1,1,0])
```

In [3]:
```
fig = plt.figure()
plt.scatter(x_1, x_2, c=y, s=60, cmap=plt.cm.bwr, edgecolors="k")
plt.grid(linestyle="dotted")
plt.xlabel("$x_1$",fontsize=18)
plt.ylabel("$x_2$",fontsize=18)
plt.show()
```



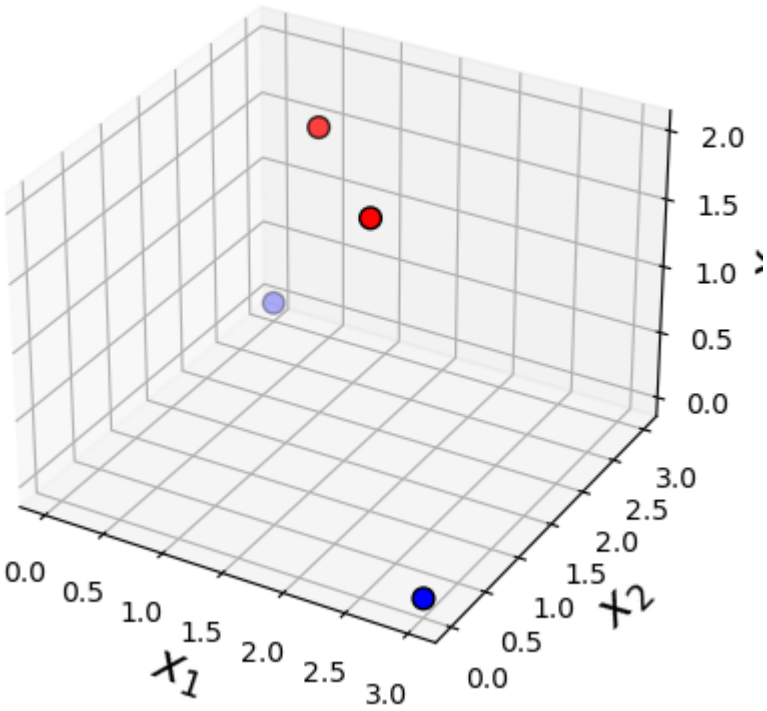
Por ejemplo, $K(x_1, x_2) = (x_1, x_2, x_1 \cdot x_2)$

In [4]:
```
def kernel(x_1, x_2):
    X = np.c_[(x_1, x_2)]
    x_1 = X[:,0]
    x_2 = X[:,1]
    x_3 = x_1 * x_2
    return np.array([x_1, x_2, x_3])
```

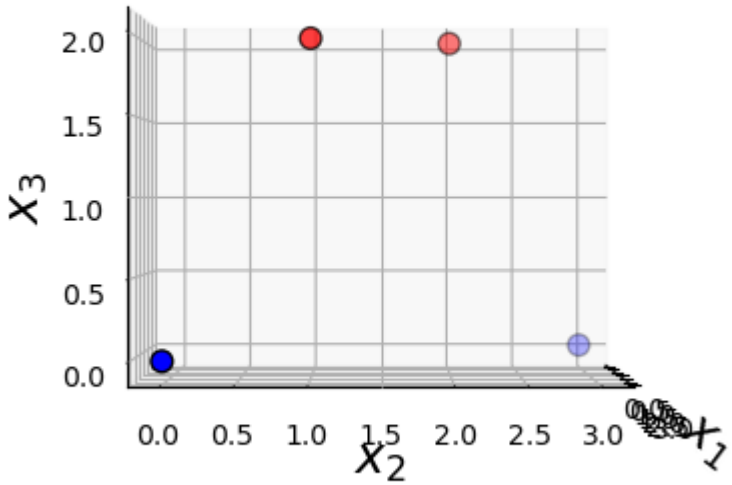
In [5]:
```
K  = kernel(x_1, x_2)
K
```

Out[5]:
```
array([[0, 1, 2, 3],
       [3, 2, 1, 0],
       [0, 2, 2, 0]])
```

In [6]:
```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(K[0], K[1], K[2], c=y, s=60, cmap=plt.cm.bwr, edgecolors="k")
ax.set_xlabel("$x_1$",fontsize=18)
ax.set_ylabel("$x_2$",fontsize=18)
ax.set_zlabel("$x_3$",fontsize=18)
plt.show()
```

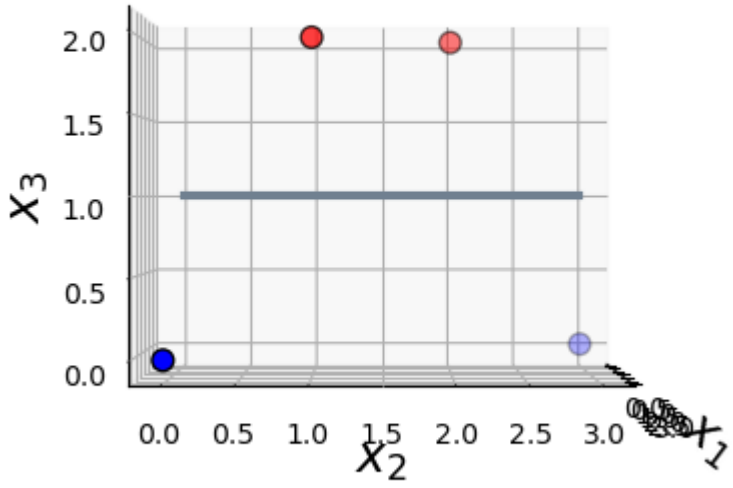


In [7]:
```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(K[0], K[1], K[2], c=y, s=60, cmap=plt.cm.bwr, edgecolors="k")
ax.view_init(0, 0)
ax.set_xlabel("$x_1$",fontsize=18)
ax.set_ylabel("$x_2$",fontsize=18)
ax.set_zlabel("$x_3$",fontsize=18)
plt.show()
```

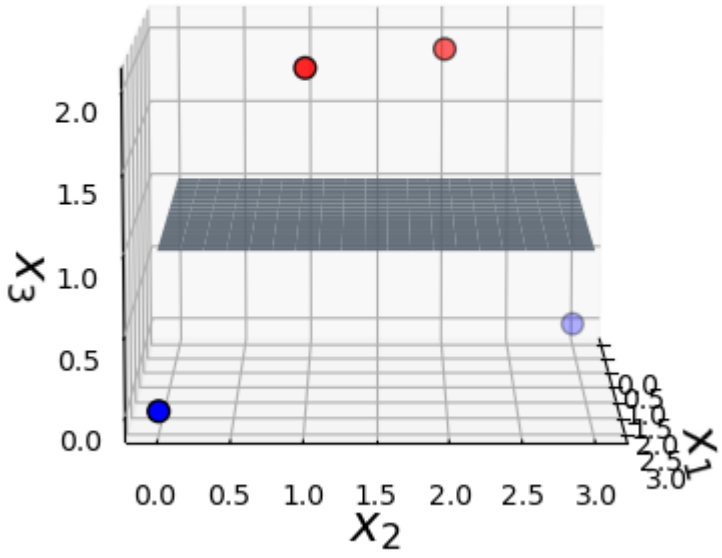


In [8]:
```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(K[0], K[1], K[2], c=y, s=60, cmap=plt.cm.bwr, edgecolors="k")
ax.plot([0,0], [0,3], [1,1], color='slategrey', lw=3)
```

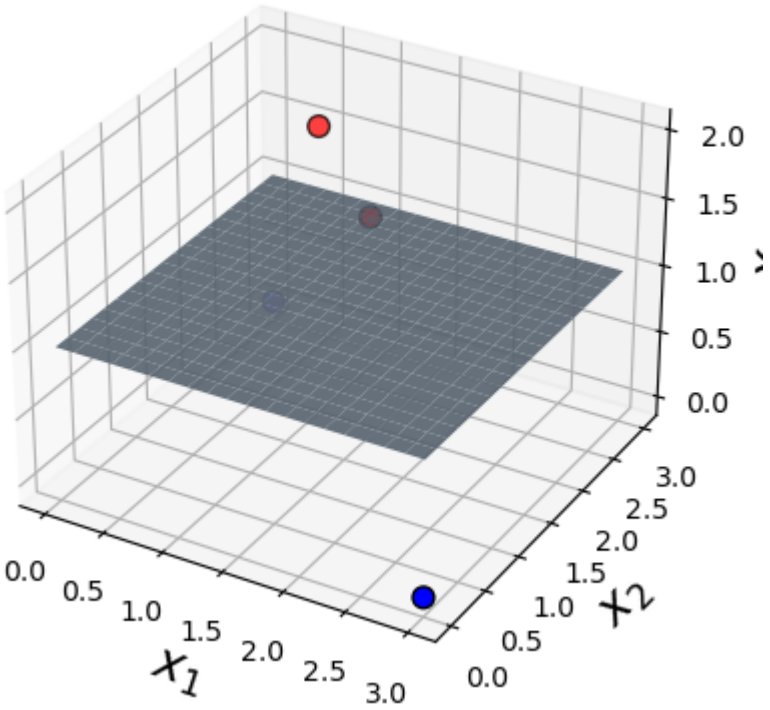
```
ax.view_init(0, 0)
ax.set_xlabel("$x_1$",fontsize=18)
ax.set_ylabel("$x_2$",fontsize=18)
ax.set_zlabel("$x_3$",fontsize=18)
plt.show()
```



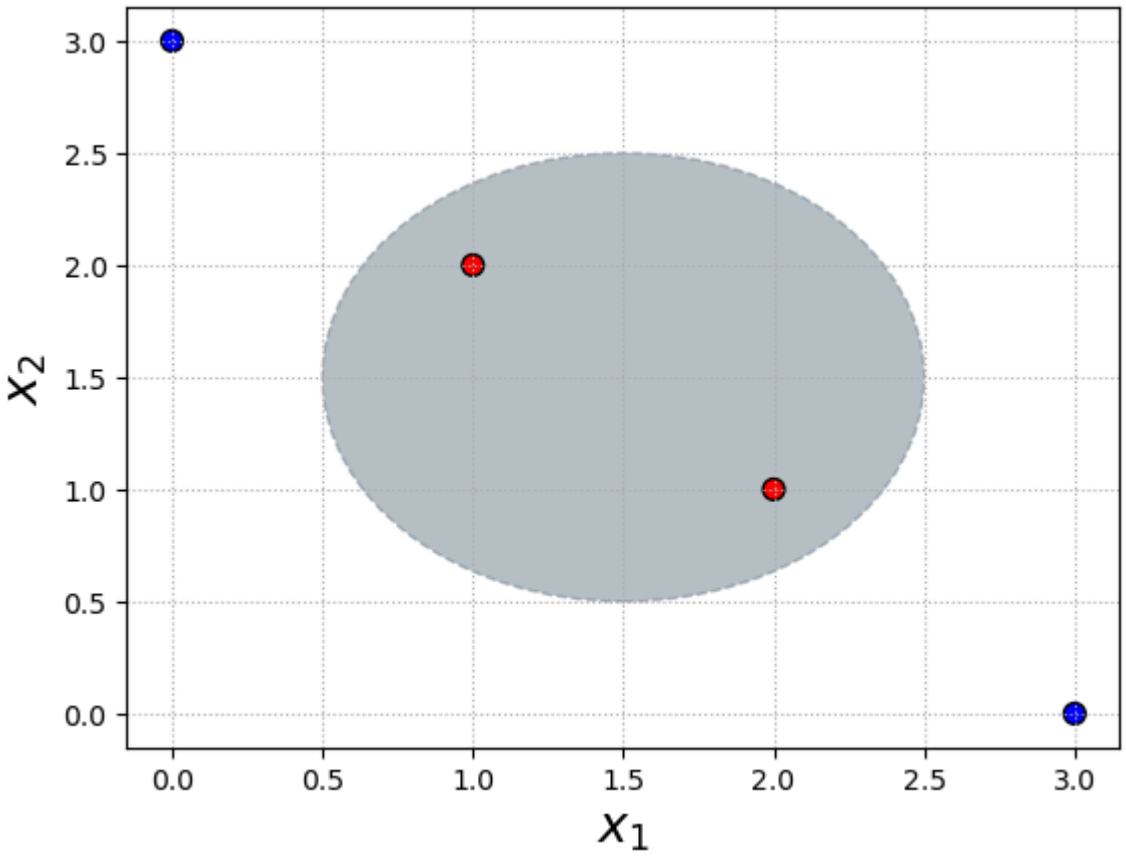
In [9]:
```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(K[0], K[1], K[2], c=y, s=60, cmap=plt.cm.bwr, edgecolors="k")
ax.view_init(10,0)
x1_lim = [0,3]
x2_lim = [0,3]
x1s = np.linspace(x1_lim[0], x1_lim[1], 20)
x2s = np.linspace(x2_lim[0], x2_lim[1], 20)
x1, x2 = np.meshgrid(x1s, x2s)
ax.plot_surface(x1s, x2, np.zeros_like(x1)+1, color="slategrey", alpha=0.9)
ax.set_xlabel("$x_1$",fontsize=18)
ax.set_ylabel("$x_2$",fontsize=18)
ax.set_zlabel("$x_3$",fontsize=18)
plt.show()
```



In [10]:
```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(K[0], K[1], K[2], c=y, s=60, cmap=plt.cm.bwr, edgecolors="k")
x1_lim = [0,3]
x2_lim = [0,3]
x1s = np.linspace(x1_lim[0], x1_lim[1], 20)
x2s = np.linspace(x2_lim[0], x2_lim[1], 20)
x1, x2 = np.meshgrid(x1s, x2s)
ax.plot_surface(x1s, x2, np.zeros_like(x1)+1, color="slategrey", alpha=0.9)
ax.set_xlabel("$x_1$",fontsize=18)
ax.set_ylabel("$x_2$",fontsize=18)
ax.set_zlabel("$x_3$",fontsize=18)
plt.show()
```



In [11]:
```
fig = plt.figure()
ax = plt.gca()
circle = plt.Circle((1.5, 1.5), 1, color='slategrey', fill='k', alpha=0.5, linestyl
ax.add_patch(circle)
ax.scatter(x_1, x_2, c=y, s=60, cmap=plt.cm.bwr, edgecolors="k")
plt.grid(linestyle="dotted")
plt.xlabel("$x_1$",fontsize=18)
plt.ylabel("$x_2$",fontsize=18)
plt.show()
```



Tipos de Kernel

- linear
- polynomial
- rbf
- otros..

In [12]:
```
from sklearn import svm
from sklearn.inspection import DecisionBoundaryDisplay
```

In [13]:
```
X = np.c_[(x_1, x_2)]
clf = svm.SVC(kernel="linear").fit(X, y)
```

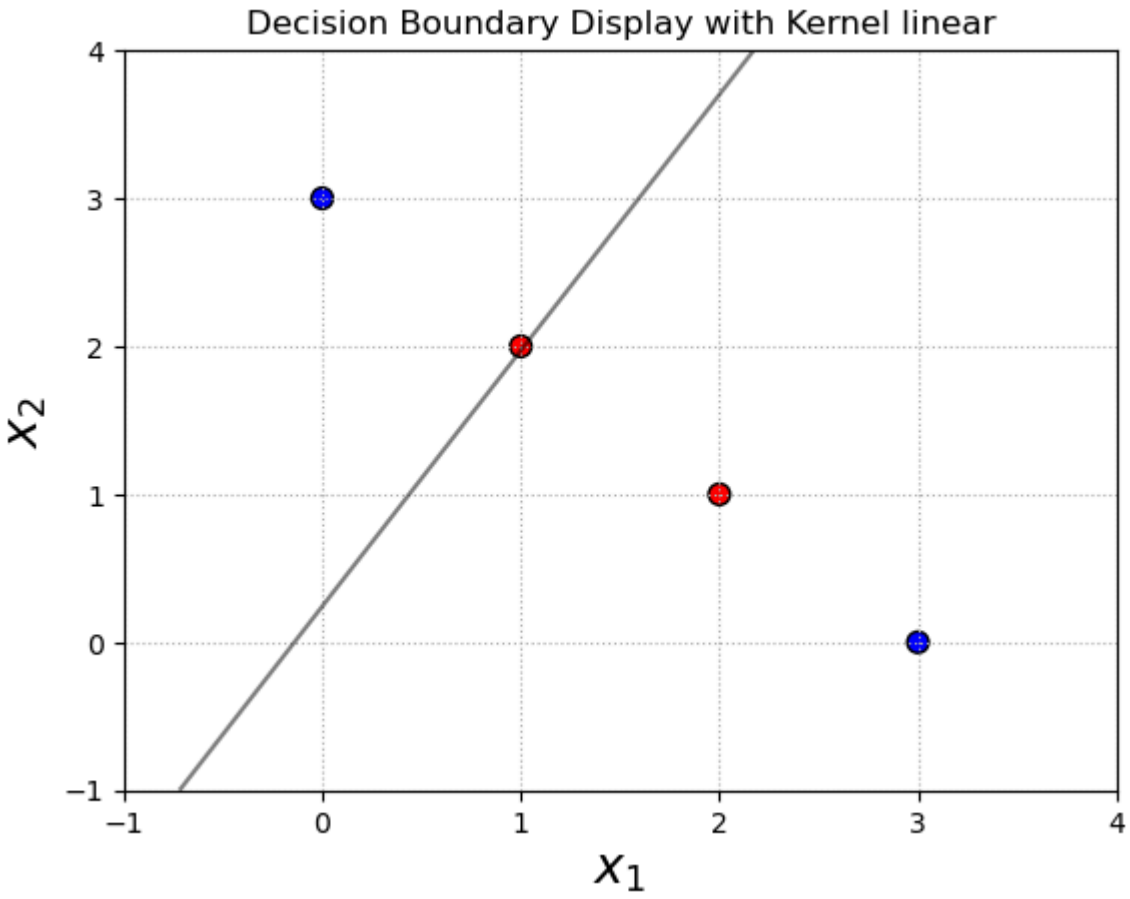
In [14]:
```
def plot_boundary(X,y, kernel='rbf'):
    fig = plt.figure()
    if kernel=="linear":
        model = svm.LinearSVC(dual=True).fit(X, y)
    else:
        model = svm.SVC(kernel=kernel).fit(X, y)
    plt.scatter(X[:,0], X[:,1], c=y, s=60, cmap=plt.cm.bwr, edgecolors="k")
```

```
    ax = plt.gca()
    DecisionBoundaryDisplay.from_estimator(model, X, plot_method="contour", colors=
    plt.grid(linestyle="dotted")
    plt.xlabel("$x_1$",fontsize=18)
    plt.ylabel("$x_2$",fontsize=18)
    plt.title(f"Decision Boundary Display with Kernel {kernel}")
    plt.show()
```

Kernel Lineal

$$\mathbf{w} \cdot \mathbf{x} + b$$

In [15]:
```
plot_boundary(X,y, kernel='linear')
```

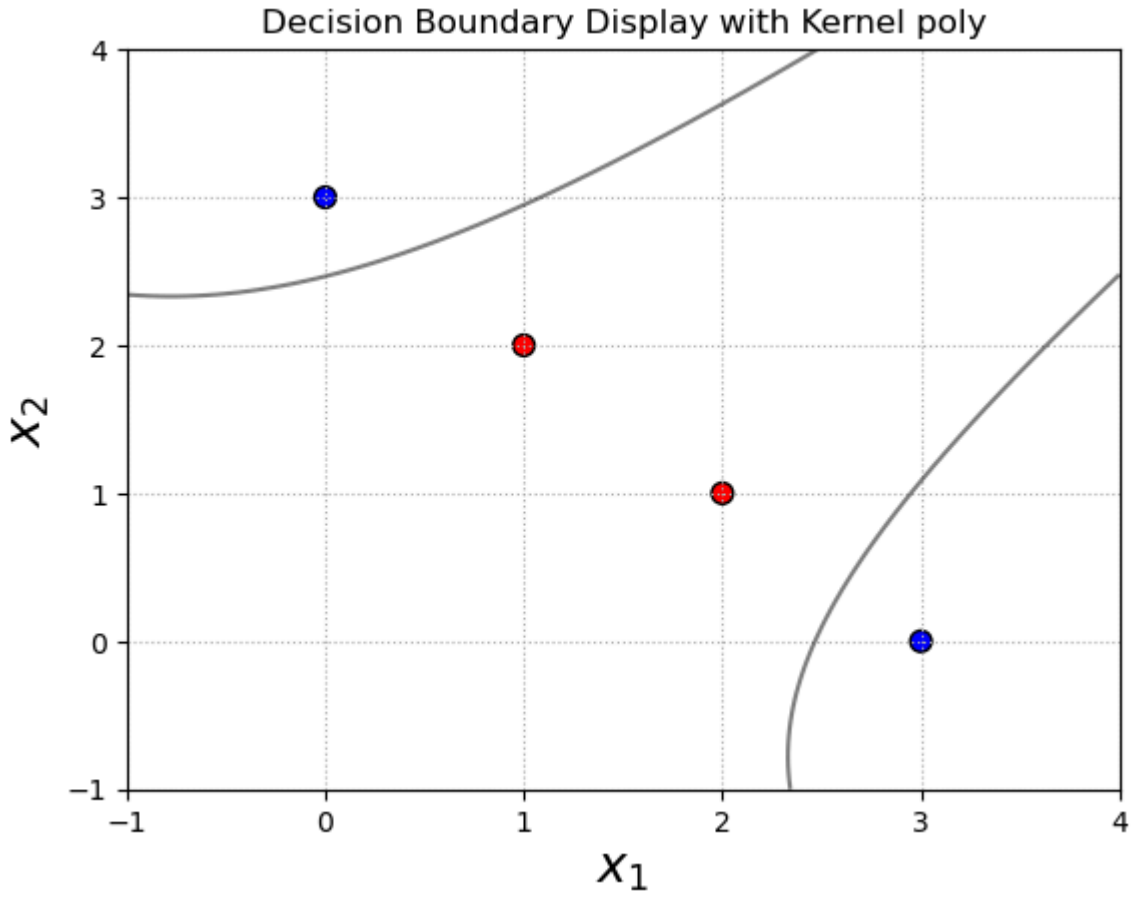


Kernel Polinomial

$$(\mathbf{w} \cdot \mathbf{x} + b)^p$$

donde p es el grado del polinomio.

In [16]:
```
plot_boundary(X,y, kernel="poly")
```

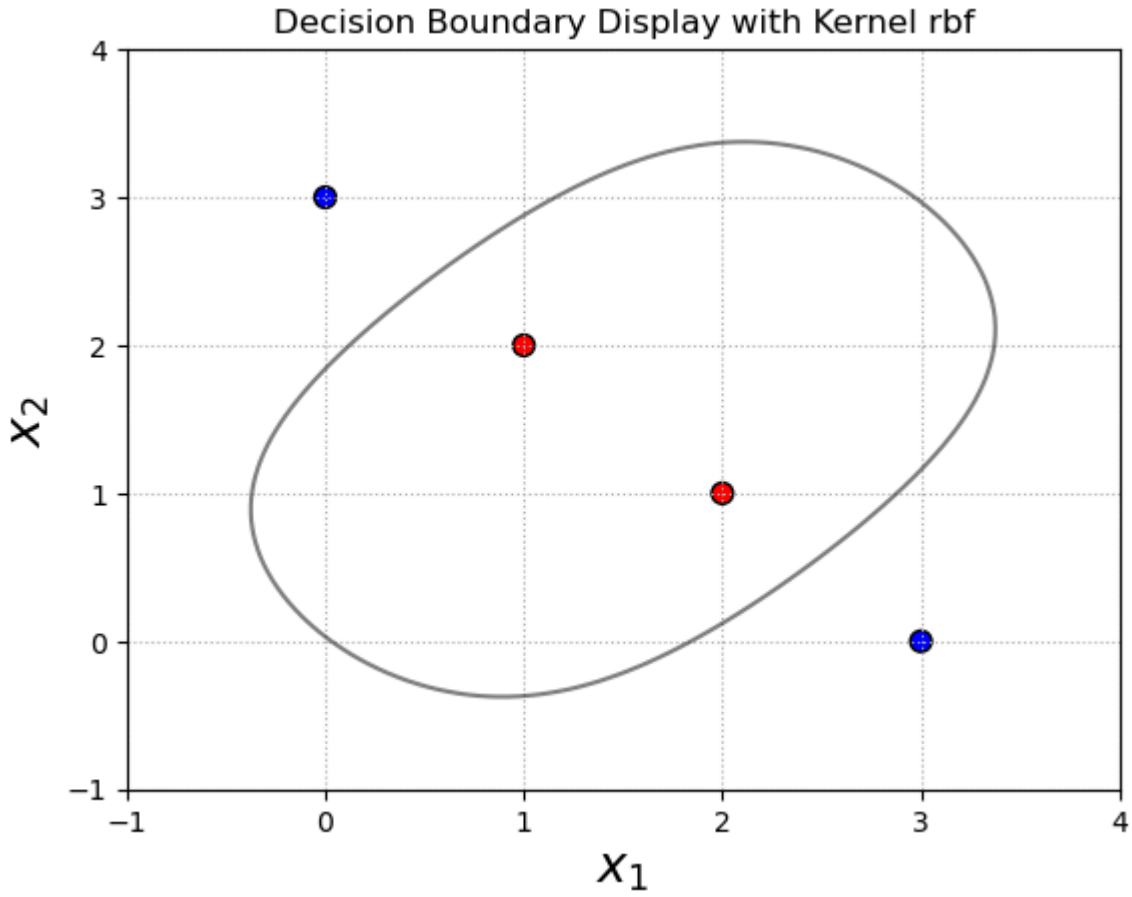


Kernel RBF

$$e^{-\gamma\|\mathbf{x}\|^2}$$

donde γ define hasta dónde llega la influencia de un solo ejemplo de entrenamiento, donde los valores bajos significan "lejos" y los valores altos significan "cerca". Por lo tanto, cuando gamma es muy grande se puede caer en sobreajuste porque captura demasiado bien la "forma" de los datos. En caso contrario, cuando gamma es pequeño, el modelo está demasiado restringido y no ajusta lo suficiente (underfitting).

In [17]:
```
plot_boundary(X,y, kernel='rbf')
```

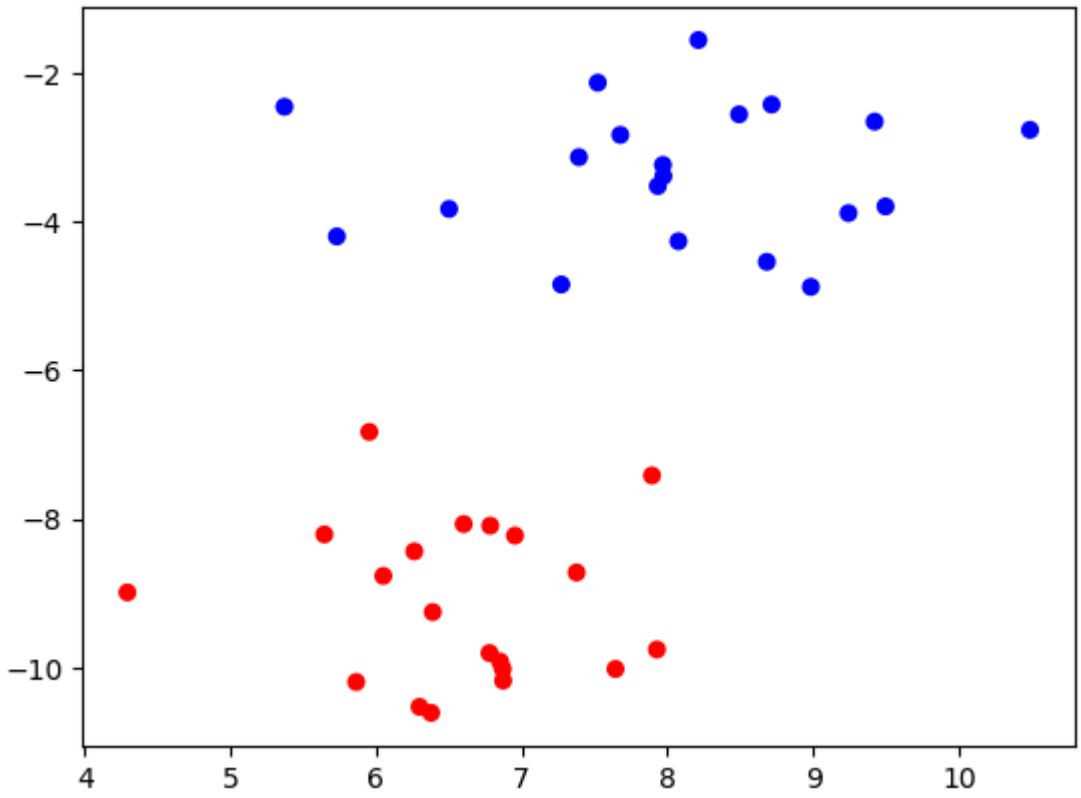


Support Vector Machines

Se basan en el principio del margen máximo que separa el hiperplano.

In [18]:
```
from sklearn.datasets import make_blobs, make_moons
```

In [19]:
```
X, y = make_blobs(n_samples=40, centers=2, random_state=6)
plt.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=plt.cm.bwr)
plt.show()
```



In [20]:
```
clf = svm.SVC(kernel="linear", C=1000)
clf.fit(X, y)
```

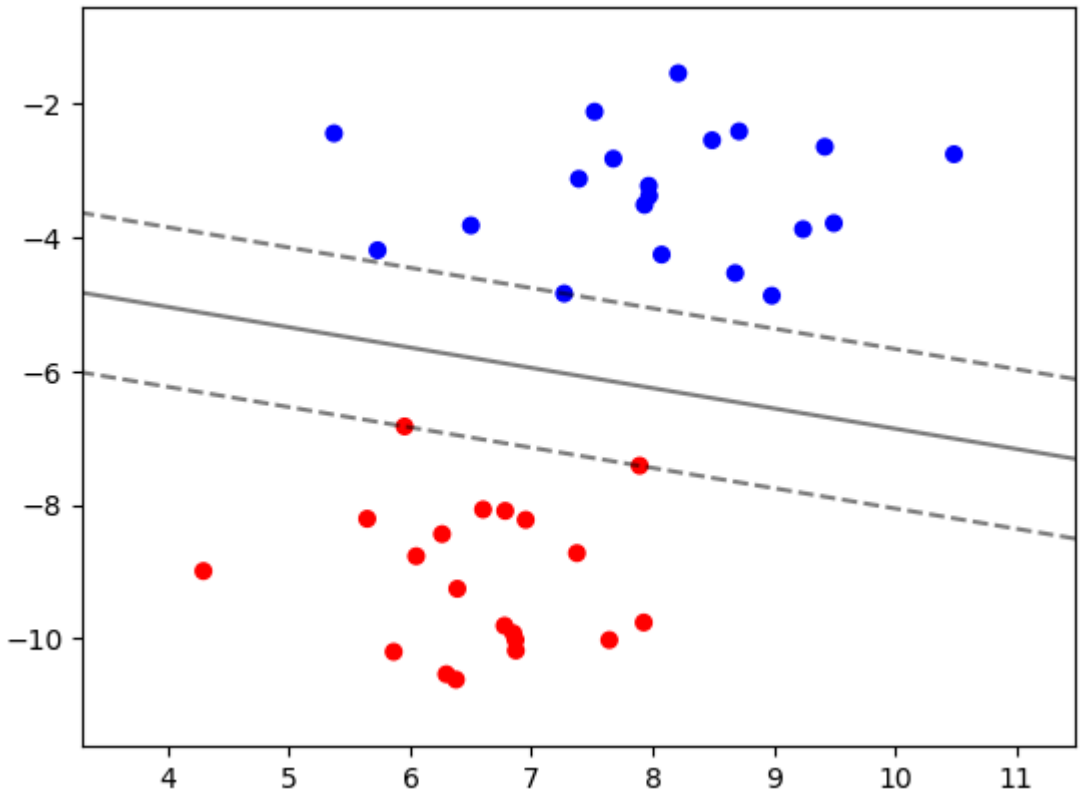
Out[20]:

SVC

SVC(C=1000, kernel='linear')

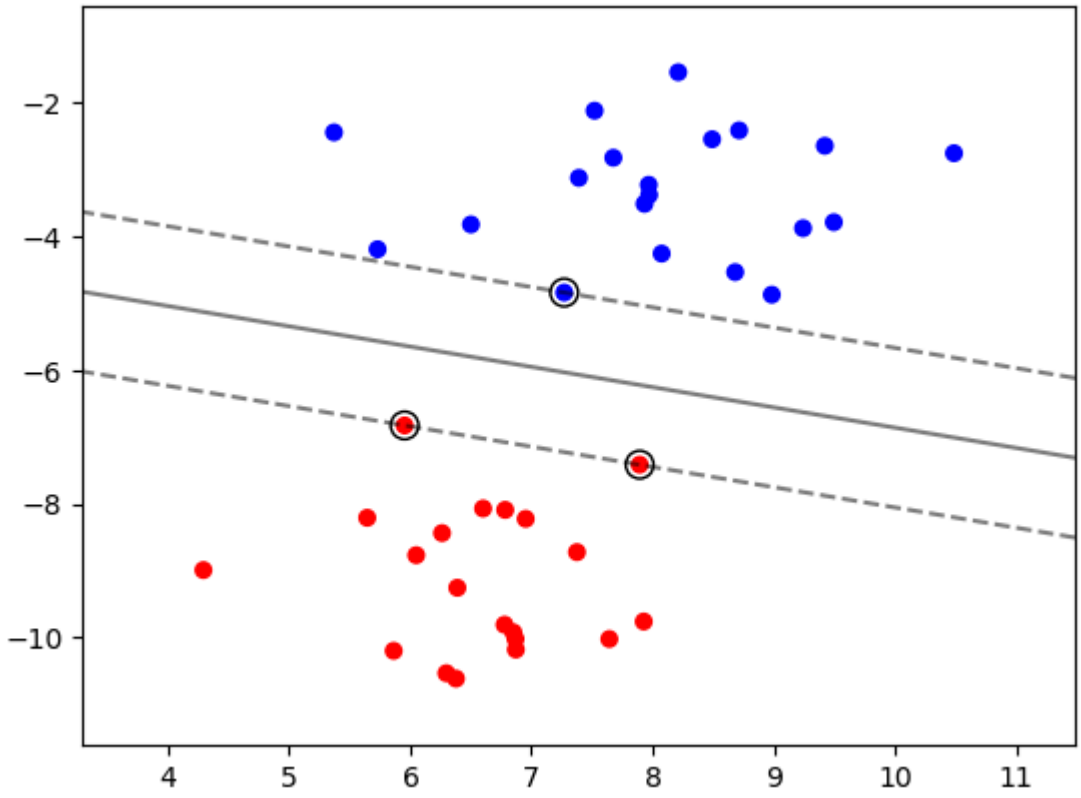
Hiperplano de máximo margen

In [21]:
```
plt.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=plt.cm.bwr)
ax = plt.gca()
DecisionBoundaryDisplay.from_estimator(clf, X, plot_method="contour", colors="k", l
    alpha=0.5, linestyles=["--", "-", "--"], ax=ax)
plt.show()
```



Vectores de soporte

In [22]:
```
plt.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=plt.cm.bwr)
ax = plt.gca()
DecisionBoundaryDisplay.from_estimator(clf, X, plot_method="contour", colors="k", l
    alpha=0.5, linestyles=["--", "-", "--"], ax=ax)
ax.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1], s=100, linewidth
plt.show()
```

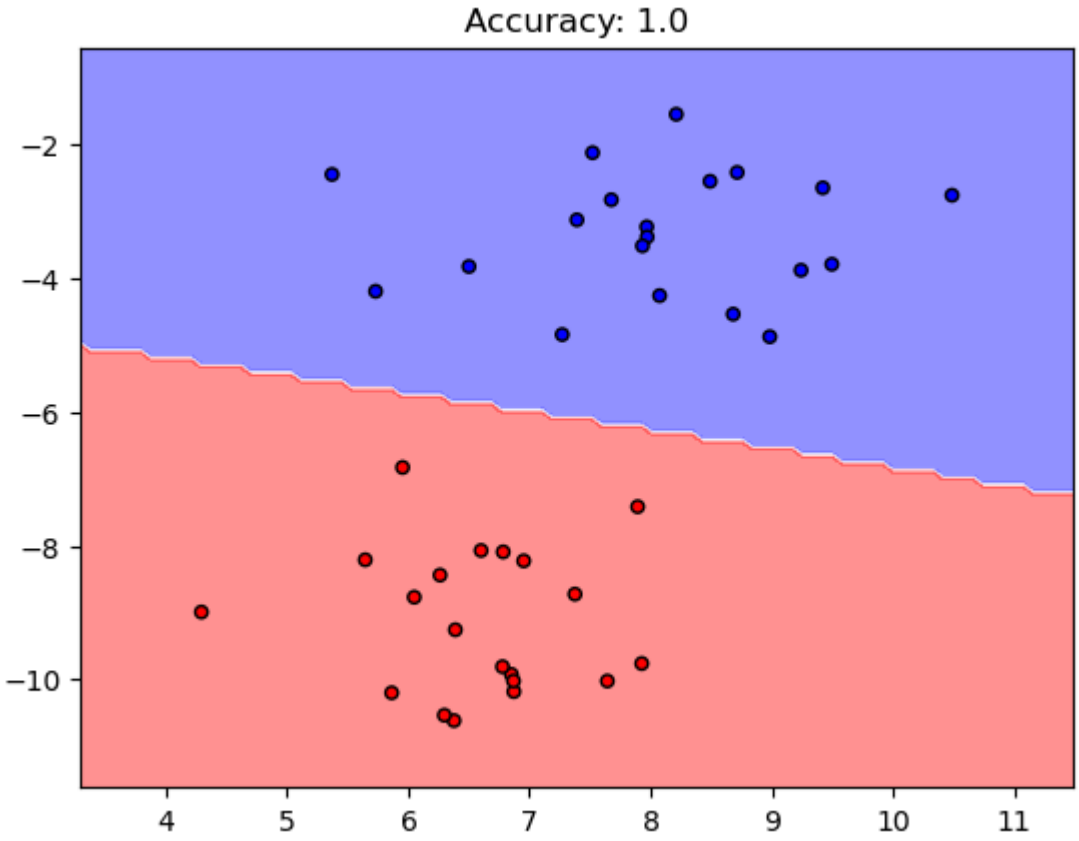


Support Vector Classifier

In [23]:
```
from sklearn.metrics import accuracy_score
```

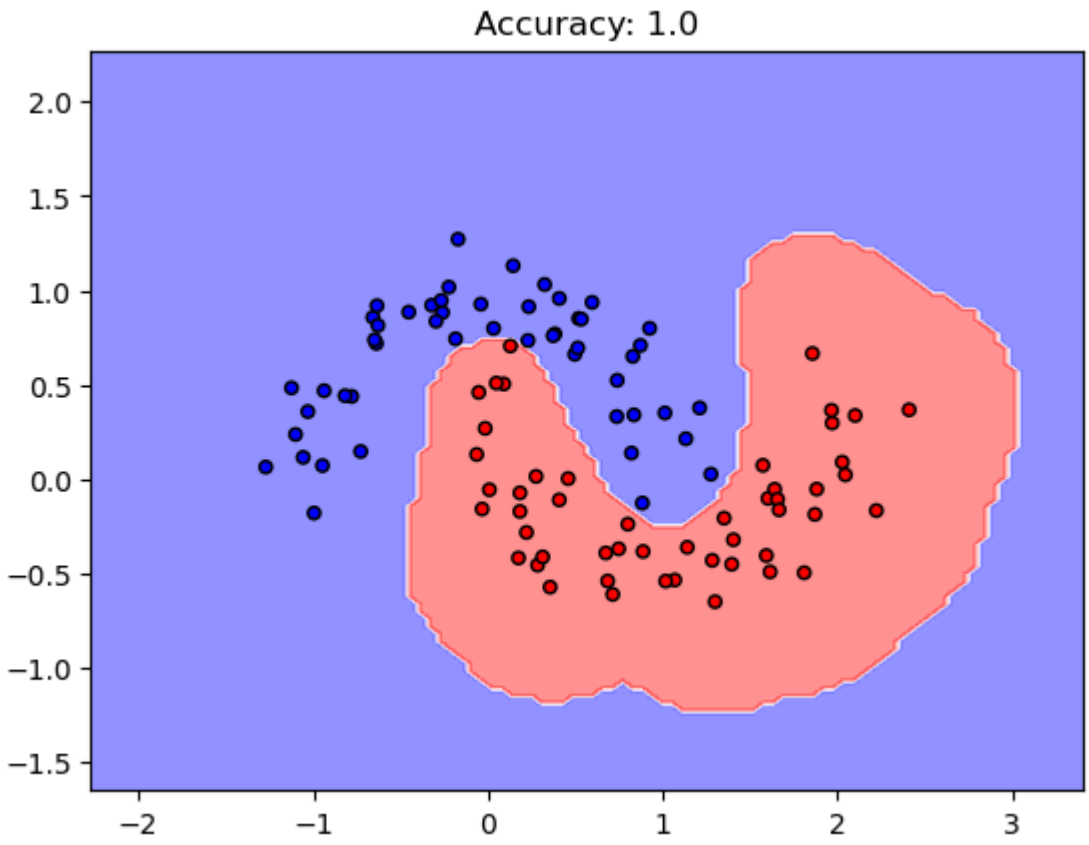
In [24]:
```
def plot_svc(X,y, gamma='scale',C=1):
    SVM = svm.SVC(gamma=gamma, C=C).fit(X,y)
    disp = DecisionBoundaryDisplay.from_estimator(SVM, X, response_method="predict"
    disp.ax_.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.bwr, s=20, edgecolors="k")
    plt.title(f"Accuracy: {round(accuracy_score(y, SVM.predict(X)),2)}")
    plt.show()
```

In [25]:
```
plot_svc(X,y)
```



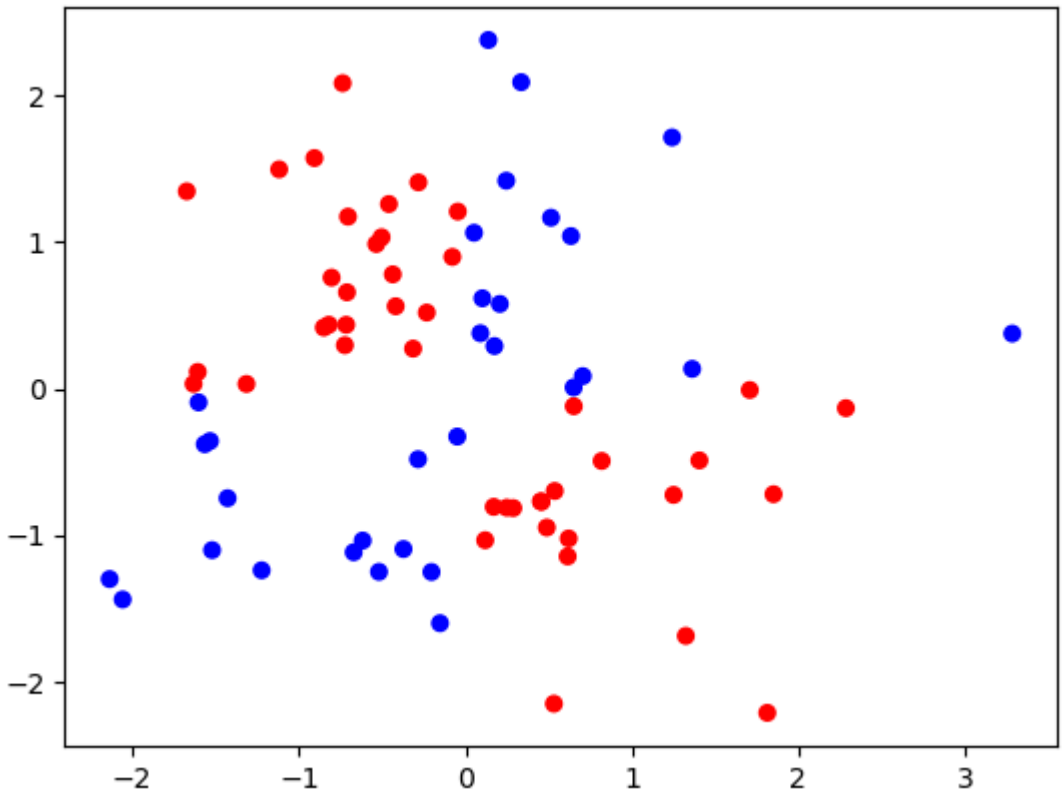
In [26]:
```
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
```

In [27]:
```
plot_svc(X,y,gamma=10,C=100)
```

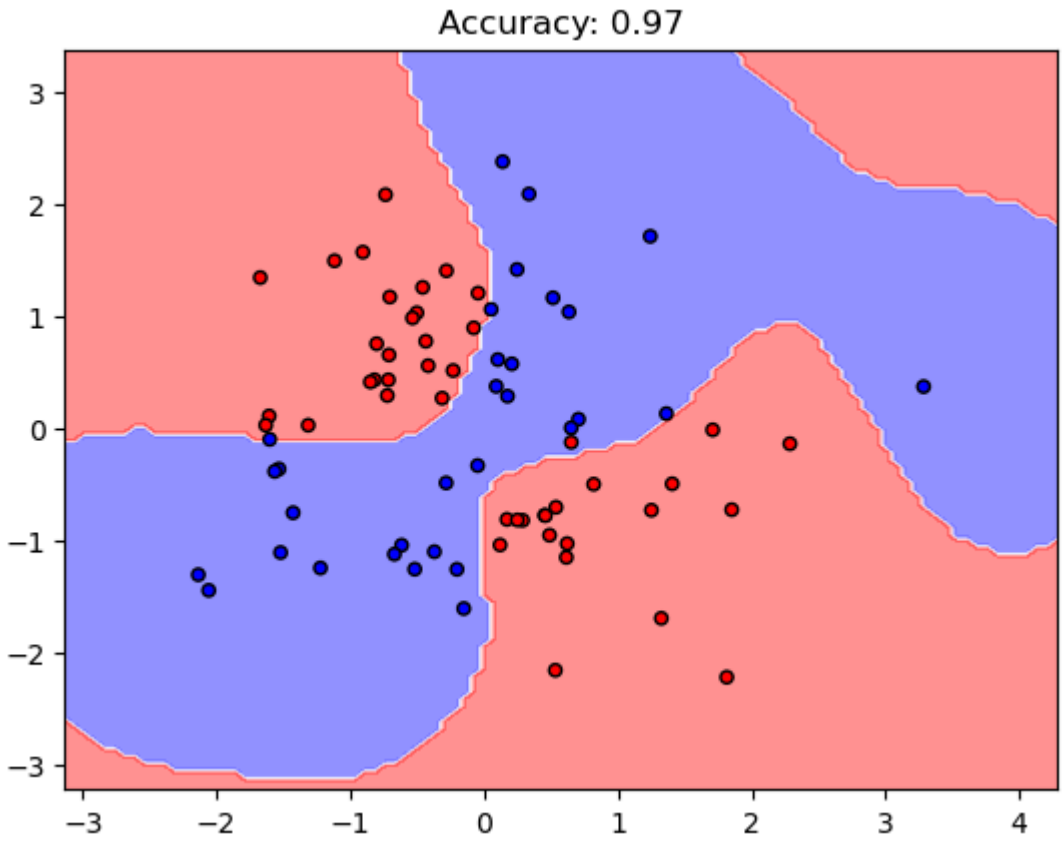


In [28]:
```
def non_separable(nsize=400, random_state=11238):
    np.random.seed(random_state)
    x_xor = np.random.randn(nsize, 2)
    y_xor = np.logical_xor(x_xor[:, 0] > 0, x_xor[:, 1] > 0)
    y_xor = np.where(y_xor, 1, -1)
    return x_xor, y_xor
```

In [29]:
```
X, y = non_separable(nsize=75)
plt.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=plt.cm.bwr)
plt.show()
```

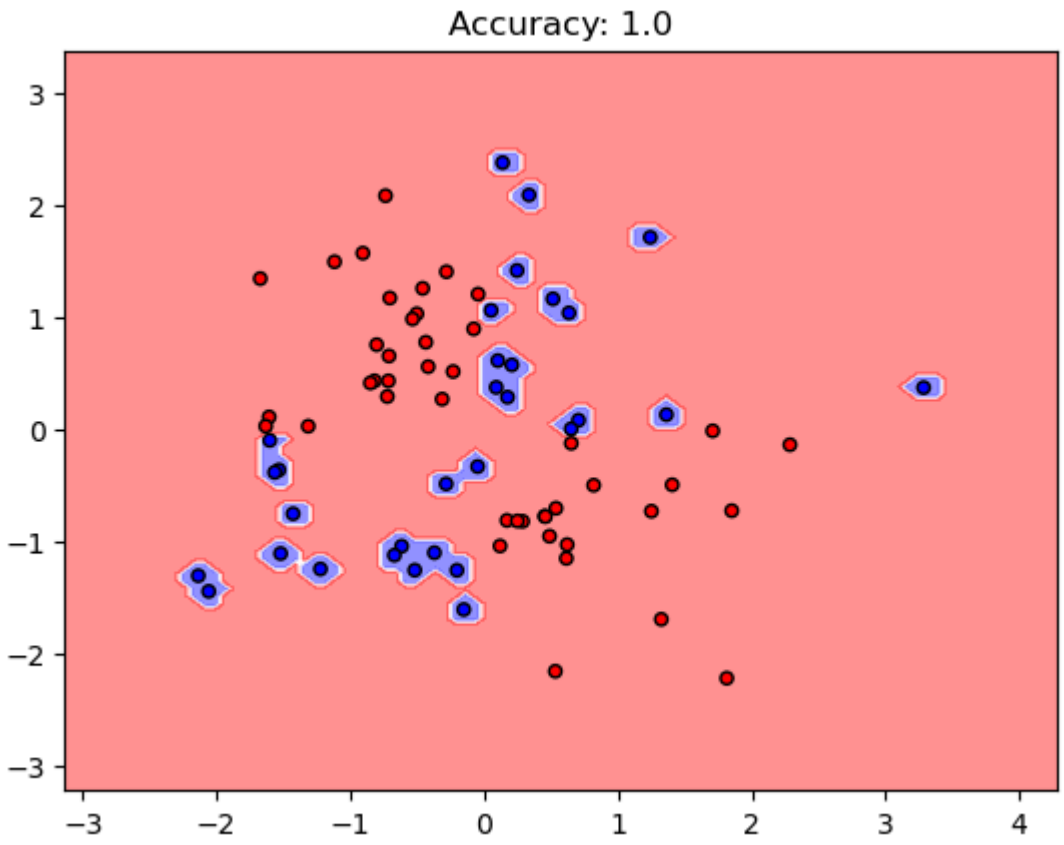


In [30]:
```
plot_svc(X,y, gamma=1)
```



Overfitting!

In [31]:
```
plot_svc(X,y, gamma=100)
```



Support Vector Regression

El método Support Vector Classifier depende solo de un subconjunto de los datos de entrenamiento, porque la función de costo para construir el modelo no se preocupa por los puntos de entrenamiento que se encuentran más allá del margen. Análogamente, se puede se puede extender este modelo para resolver problemas de regresión.

In [32]:
```
X = [[0, 0], [2, 2]]
y = [0.5, 2.5]
regr = svm.SVR().fit(X, y)
regr.predict([[1, 1]])
```

Out[32]:
```
array([1.5])
```

In [33]:
```
X = np.sort(5 * np.random.rand(40, 1), axis=0)
y = np.sin(X).ravel()
y[::5] += 3 * (0.5 - np.random.rand(8))
```

In [34]:
```
from sklearn.svm import SVR
```

In [35]:
```
svr_rbf = SVR(kernel="rbf", C=100, gamma=0.1, epsilon=0.1)
svr_lin = SVR(kernel="linear", C=100, gamma="auto")
svr_poly = SVR(kernel="poly", C=100, gamma="auto", degree=3, epsilon=0.1, coef0=1)
```

In [36]:
```
def plot_svr(X,y):
    lw = 2
    svrs = [svr_rbf, svr_lin, svr_poly]
    kernel_label = ["RBF", "Linear", "Polynomial"]
    model_color = ["m", "c", "g"]
    fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(18, 6), sharey=True)
```

```
    for ix, svr in enumerate(svrs):
        axes[ix].plot(
            X,
            svr.fit(X, y).predict(X),
            color=model_color[ix],
            lw=lw,
            label="{} model".format(kernel_label[ix]),
        )
        axes[ix].scatter(
            X[svr.support_],
            y[svr.support_],
            facecolor="none",
            edgecolor=model_color[ix],
            s=50,
            label="{} support vectors".format(kernel_label[ix]),
        )
        axes[ix].scatter(
            X[np.setdiff1d(np.arange(len(X)), svr.support_)],
            y[np.setdiff1d(np.arange(len(X)), svr.support_)],
            facecolor="none",
            edgecolor="k",
            s=50,
            label="other training data",
        )
        axes[ix].legend(
            loc="upper center",
            bbox_to_anchor=(0.5, 1.1),
            ncol=1,
            fancybox=True,
            shadow=True,
        )
    fig.text(0.5, 0.04, "data", ha="center", va="center")
    fig.text(0.06, 0.5, "target", ha="center", va="center", rotation="vertical")
    fig.suptitle("Support Vector Regression", fontsize=14)
    plt.show()
```

In [37]:
```
plot_svr(X,y)
```

